

GameplayEffectExecutionCalculation (简称 **ExecCalc**) 是 GAS 中用于处理**复杂数值逻辑**的核心机制。当简单的加减乘除 (Modifier) 无法满足需求时 (例如: 需要根据“暴击率”判断是否暴击, 或者根据“防御力”和“穿透值”计算最终伤害), 就需要使用它。

基于你提供的 ExecCalc_Damage.cpp 和 ExecCalc_Damage.h, 以下是关于自定义执行计算的标准流程、代码深度解析及关键注意事项。

一、标准流程 (Standard Workflow)

实现一个自定义伤害计算通常分为以下 4 个步骤:

1. 定义捕获属性 (Capture Definitions):

- 你需要在这个计算中“读取”谁的什么属性? (例如: 读取目标的防御力、格挡率, 读取**来源**的攻击力)。
- 使用宏来声明和定义这些捕获项。

2. 构造函数注册 (Constructor Registration):

- 告诉 GAS 系统, 当这个 GE 运行时, 必须把上面定义的属性值抓取过来准备好。

3. 执行核心逻辑 (Execute Implementation):

- 重写 Execute_Implementation 函数。
- 获取上下文 (谁打谁)。
- 获取基础数值 (比如通过 SetByCaller 传入的基础伤害)。
- 计算属性值 (计算经其他 GE 修改后的最终防御力/格挡率)。
- 执行自定义数学逻辑 (判定格挡、计算减伤)。

4. 输出结果 (Output):

- 将计算后的最终结果 (Final Damage) 写入到一个属性中 (通常是 Meta Attribute, 如 IncomingDamage)。
-

二、代码详细解析

1. 准备捕获属性: Struct AuraDamageStatics

这是一种标准的 **Singleton (单例)** 写法, 用于集中管理该计算类需要捕获的所有属性定义。

C++

// 制作所捕获的属性变量

struct AuraDamageStatics

{

// 1. 声明：我们需要捕获 Armor 和 BlockChance

DECLARE_ATTRIBUTE_CAPTUREDEF(Armor)

DECLARE_ATTRIBUTE_CAPTUREDEF(BlockChance)

AuraDamageStatics()

{

// 2. 定义：具体怎么捕获？

// 参数含义：(属性集类, 属性名, 目标 Source/Target, 是否快照 Snapshot)

// 捕获目标的 Armor 属性, 不使用快照 (false)

DEFINE_ATTRIBUTE_CAPTUREDEF(UAuraAttributeSet,Armor,Target,false);

// 捕获目标的 BlockChance 属性, 不使用快照 (false)

DEFINE_ATTRIBUTE_CAPTUREDEF(UAuraAttributeSet,BlockChance,Target,false);

}

};

// 静态访问函数, 确保只初始化一次

static const AuraDamageStatics& DamageStatics()

{

static AuraDamageStatics DStatics;

return DStatics;

}

- **Target vs Source:** 这里填 Target, 因为护甲和格挡率是**挨打的那一方 (目

标) **的属性。如果是计算攻击方的攻击力, 则填 Source。

- **Snapshot (快照):** 这里填 false。
 - true (Snapshot): GE **创建时**就把属性值存下来 (比如火球飞出去的一瞬间)。
 - false (Not Snapshot): GE **应用时**才去读取属性值 (比如火球砸到脸上的一瞬间)。
 - **要点:** 对于伤害计算, 通常使用 false, 因为我们希望计算的是命中那一刻敌人的防御状态。

2. 注册捕获: 构造函数

C++

```
UExecCalc_Damage::UExecCalc_Damage()
{
    // 告诉 GAS: "我要用到这两个属性, 执行前帮我准备好数据"
    RelevantAttributesToCapture.Add(DamageStatics().ArmorDef);
    RelevantAttributesToCapture.Add(DamageStatics().BlockChanceDef);
}
```

3. 核心逻辑: Execute_Implementation

这是逻辑发生的地方。

A. 获取上下文与 Tag

C++

```
const UAbilitySystemComponent* SourceASC =
    ExecutionParams.GetSourceAbilitySystemComponent();

const UAbilitySystemComponent* TargetASC =
    ExecutionParams.GetTargetAbilitySystemComponent();

// ... 获取 AvatarActor ...

const FGameplayEffectSpec& Spec = ExecutionParams.GetOwningSpec();
```

```
// 获取 Source 和 Target 身上的所有 GameplayTag  
// 这通常用于判断: "如果目标有'易伤'Tag, 伤害翻倍" 这种逻辑
```

```
const FGameplayTagContainer* SourceTag =  
Spec.CapturedSourceTags.GetAggregatedTags();
```

```
const FGameplayTagContainer* TargetTag =  
Spec.CapturedTargetTags.GetAggregatedTags();
```

```
FAggregatorEvaluateParameters EvaluateParameters;
```

```
EvaluateParameters.SourceTags = SourceTag;
```

```
EvaluateParameters.TargetTags = TargetTag;
```

B. 获取基础伤害 (SetByCaller)

```
C++
```

```
// Get Damage Set by Caller Magnitude
```

```
// 获取我们在 Ability 中通过 "Assign Tag Set By Caller Magnitude" 传入的基础数值
```

```
float Damage = Spec.GetSetByCallerMagnitude(FAuraGameplayTags::Get().Damage);
```

- 这非常关键。伤害的基础值（比如武器造成 50 点伤害）不是从属性里读的，而是从 Ability 里传过来的。

C. 计算捕获的属性值 (BlockChance)

```
C++
```

```
float TargetBlockChance = 0.f;
```

```
// 计算捕获的属性值 Magnitude
```

```
// 这一步会考虑目标身上所有的 Modifier（比如装备加成、Buff 加成）计算出最终的格挡率
```

```
ExecutionParams.AttemptCalculateCapturedAttributeMagnitude(
```

```
    DamageStatics().BlockChanceDef, // 我们要算哪个属性？
```

```
    EvaluateParameters,                // 参数（包含 Tag，用于过滤某些这就生效的  
    Modifier)
```

```
    TargetBlockChance                // 输出结果存到这里
```

```
);
// 夹值, 防止负数
TargetBlockChance = FMath::Max<float>(0.f,TargetBlockChance);
```

D. 自定义业务逻辑 (格挡判定)

```
C++

// 典型的 RNG (随机数) 逻辑, 这是 Modifier 做不到的
const bool bBlocked = FMath::RandRange(1,100) < TargetBlockChance;

// 如果格挡成功, 伤害减半
Damage = bBlocked ? (Damage/2.f) : Damage;
```

E. 输出结果

```
C++

// 1. 创建修改器数据
// 目标属性: IncomingDamage (Meta Attribute)
// 操作类型: Additive (加法) —— 这意味着如果有多个 ExecCalc, 它们的结果会累加
// 数值: 计算后的 Damage

const FGameplayModifierEvaluatedData EvaluatedData(
    UAuraAttributeSet::GetIncomingDamageAttribute(),
    EGameplayModOp::Additive,
    Damage
);

// 2. 添加到输出列表

OutExecutionOutput.AddOutputModifier(EvaluatedData);
```

- **为什么是 Additive?** 虽然通常伤害是覆盖, 但 IncomingDamage 作为一个“积压”属性, 使用 Additive 更安全。比如该 GE 同时触发了物理伤害计算和魔法伤害计算, 两者都往 IncomingDamage 里加值。

三、 关键注意要点

1. Meta Attributes 的使用

- 注意最后修改的是 IncomingDamage，而不是 Health。
- **原因：**ExecCalc 只负责**计算数值**。真正的“扣血”、“死亡判断”、“受击反应”应该留给 AttributeSet 的 PostGameplayEffectExecute 去处理。ExecCalc 把计算好的伤害扔给 IncomingDamage，AttributeSet 接手后将它从 Health 中扣除。

2. Snapshotting (快照) 的选择

- 代码中使用了 false。
- 如果你的游戏设计是“子弹飞出时锁定面板属性”（类似快照机制的技能），则需要在定义时设为 true。
- 如果设计是“命中时判定属性”（例如目标在子弹飞行期间开了无敌盾），必须设为 false。

3. SetByCaller 的灵活性

- 代码通过 Spec.GetSetByCallerMagnitude 获取基础伤害。
- 这使得这个 ExecCalc 类非常通用。不管你是火球术、平砍还是陷阱，只要你给 GE 传入了 Data.Damage 的 SetByCaller 值，都可以复用这个计算类来处理防御和格挡逻辑。

4. 头文件的宏定义

- DECLARE_ATTRIBUTE_CAPTUREDEF 等宏需要在 .h 或 .cpp 顶部正确包含，通常在 GameplayEffectExecutionCalculation.h 中，但建议参考示例代码的结构，在 struct 内部管理，保持代码整洁。

通过这种方式，你成功将**复杂的战斗公式**（格挡概率判定）从蓝图或简单的 Modifier 中剥离出来，放入了 C++ 中高效且结构化地执行。